

Secured Application Development

Installing and Configuring Service



Cybersecurity



Installing and Configuring Services



Installing and Configuring Services

In this lecture, we can look at how to set up a web server and associated services in a secure way.

1. We will start by looking at **service architecture design**: how trust boundaries impact on protocol choices.
2. Web frameworks make it easier to write safe code, and we will take a look at some common options.
3. We will look at **man-in-the-middle attacks** in more detail as well as how to defend against them.
4. We will, also, look at **denial-of-service attacks** and what developers can do to mitigate their impact.
5. We will also look at **other techniques for securing our services**: reverse proxies, SSH tunnels, host firewalls, and TCP Wrappers.
6. Finally, we will move on from web servers and look at database server security and securing the file system.



Designing the Service Architecture

- ▣ When designing an application, we must decide where services will run and how they will interact.
 - ◎ If your application is small and has a modest number of users, it may be sufficient to run it on a single host, with the web server, database, and any back-end processes all running on the same machine.
 - ◎ If your application needs to serve a higher load, you may choose to split the database and web server onto separate hosts. You may also decide on several load-balanced web servers.
- ▣ Depending on where your trust boundaries lie, you may need differing levels of security between the components.

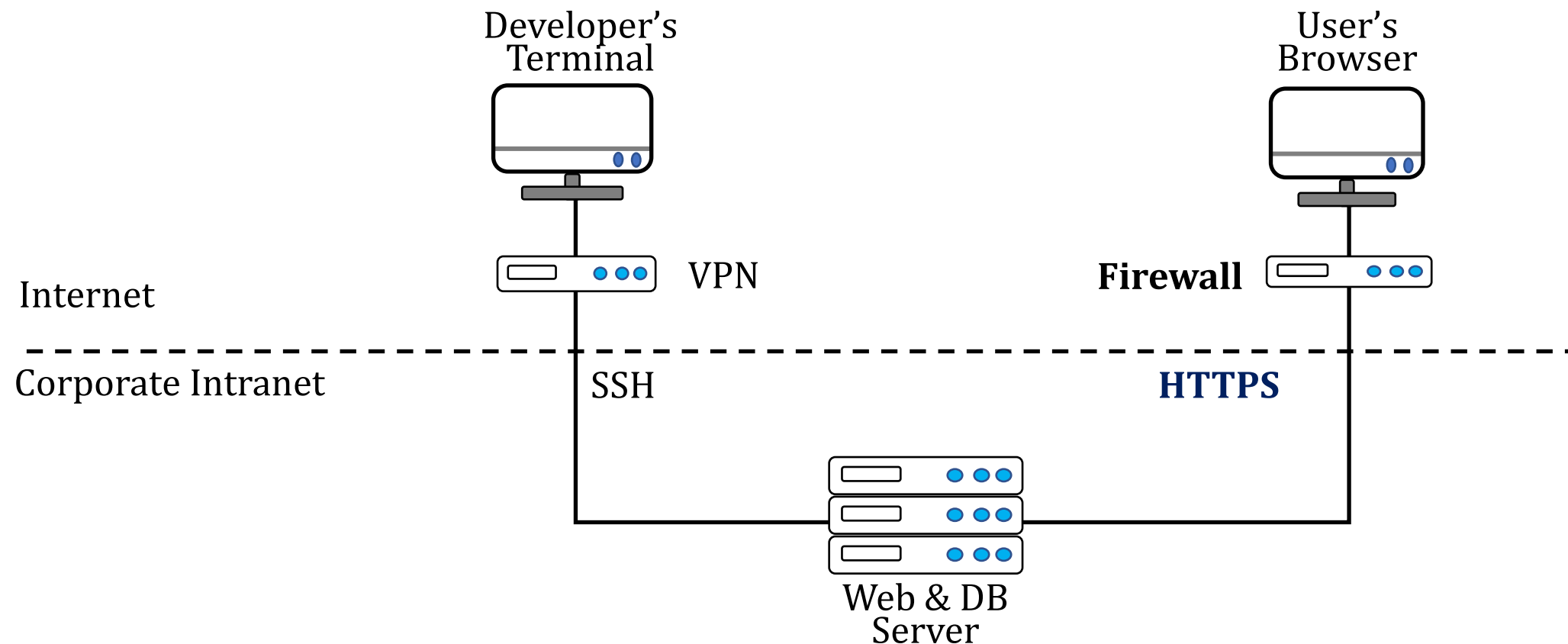


Installing and Configuring Services

Designing the Service Architecture

Let us look at a couple of example architectures and how they impact on security decisions.

- The Figure below shows a simple architecture. There is a single server running the application and database.





Designing the Service Architecture

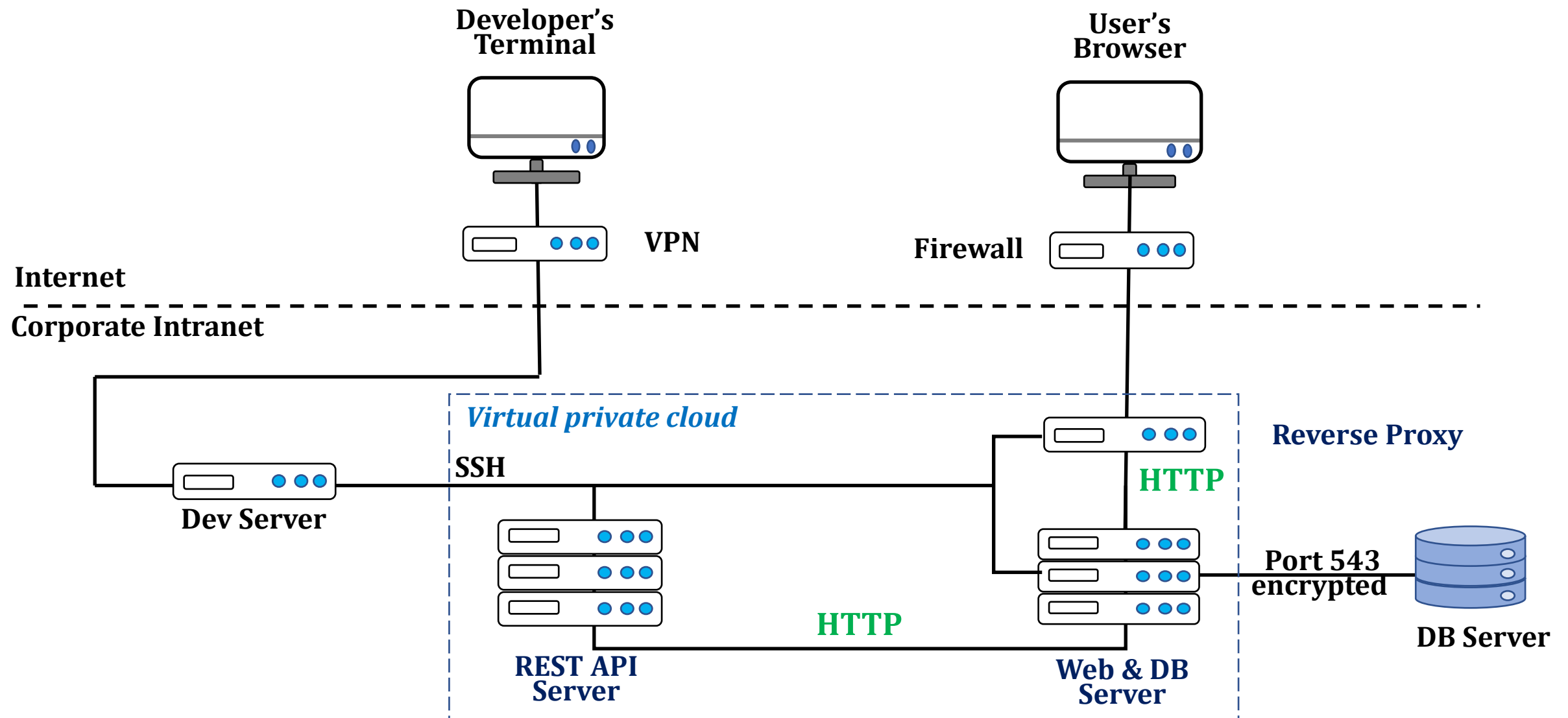
1. There is **one trust boundary**, between the Internet and the corporate intranet.
2. Since traffic **outside the intranet is untrusted**, data leaving the intranet **are encrypted**.
3. A **firewall blocks everything except** port 80 (**HTTP**) and 443 (**HTTPS**) with port 80 redirected to 443 by the web server.
4. **Developers and operators must access the host from outside the intranet. Rather than open the SSH port on the firewall**, staff access the server by connecting to the VPN, which is encrypted.



Installing and Configuring Services

Designing the Service Architecture

- The following Figure, shows a more complex architecture.





Designing the Service Architecture

- There are now **four servers**, the first three servers are:
 1. The server for the web application,
 2. a separate API server, and
 3. a reverse proxy; they are running within a virtual private cloud (VPC).
- A VPC is a virtual network with its own virtual interface, **unreachable from other hosts**.
- We do not need to encrypt HTTP traffic within the VPC **because it does not cross a trust boundary**.
- We encrypt the database connection **because the database is outside the VPC and crosses the trust boundary between it and the rest of the corporate intranet**.
- Our developers and operators need shell access on each of the servers in the VPC. **They use SSH over an externally open network interface. As this interface crosses a trust boundary, it is encrypted.**
- We can reduce the attack surface by limiting SSH to just one **development server**, **with a host firewall or TCP Wrappers**.



Web Application Frameworks

- **Web application frameworks (WAFs) provide a coherent set of tools that make developing common web functionality easier. This includes:**
 - Mapping code to URLs
 - Handling form input
 - Managing sessions
 - Authentication and authorization
 - Database integration
 - HTTP header management
 - HTML templates to combine text and data
 - Security features such as **Cross Site Request Forgery (CSRF) tokens**, escaping HTML, and sanitizing data



Web Application Frameworks

- ▣ Often, they also provide development tools for testing and debugging code. WAFs are not an absolute requirement. **The Common Gateway Interface (CGI)** has been around since the early days of HTTP.
 - ☉ **CGI provides a way for web servers to execute code, reading input from the HTTP request and sending output in the HTTP response.**
- ▣ However, writing modern applications directly on top of CGI is laborious, and developers will find themselves solving problems that have been tackled many times before.
- ▣ Using a WAF allows a developer to focus on **business logic instead of handling HTTP parameters, creating session cookies, etc.**
- ▣ Furthermore, writing these parts of the code is error-prone, and introducing vulnerabilities is easy. By using a mature, well-established framework, you leverage the work of many other developers who have already found and addressed the vulnerabilities.



Web Application Frameworks

- ▣ Ultimately, vulnerabilities come from code.
 - ◎ The more people that examine code, the less likely it is that a vulnerability will go unnoticed.
 - ◎ **The less code you write and the more you use from mature, well-scrutinized libraries, the safer your code will be.**
 - ◎ **WAFs also abstract fundamental security tasks**, such as output encoding, making it harder for us to make mistakes. **The examples is open source.** We chose it because it comes with many tools built in. Also, many third-party libraries integrate with it. This makes its ecosystem consistent and tools interoperate well.
 - ◎ **Code written with Django can be very small and clear.** This makes code easy to read, and easy-to-read code is easier to spot vulnerabilities in. Furthermore, it is a good tool for learning and exploring security techniques.



Server-side

- ☐ [Apache Wicket](#)
- ☐ [ASP.NET Core](#)
- ☐ [CakePHP](#)
- ☐ [Catalyst](#)
- ☐ [CodeIgniter](#)
- ☐ [CppCMS](#)
- ☐ [Django](#)
- ☐ [Express.js](#)
- ☐ [Flask](#)
- ☐ [Grails](#)
- ☐ [Jam.py](#)
- ☐ [Laravel](#)
- ☐ [Mojolicious](#)
- ☐ [Pop PHP Framework](#)
- ☐ [Phoenix](#)
- ☐ [Ruby on Rails](#)
- ☐ [Sails.js](#)
- ☐ [Symfony](#)
- ☐ [Spring MVC](#)
- ☐ [Wt \(web toolkit\)](#)
- ☐ [Yii](#)
- ☐ [Zend Framework](#)



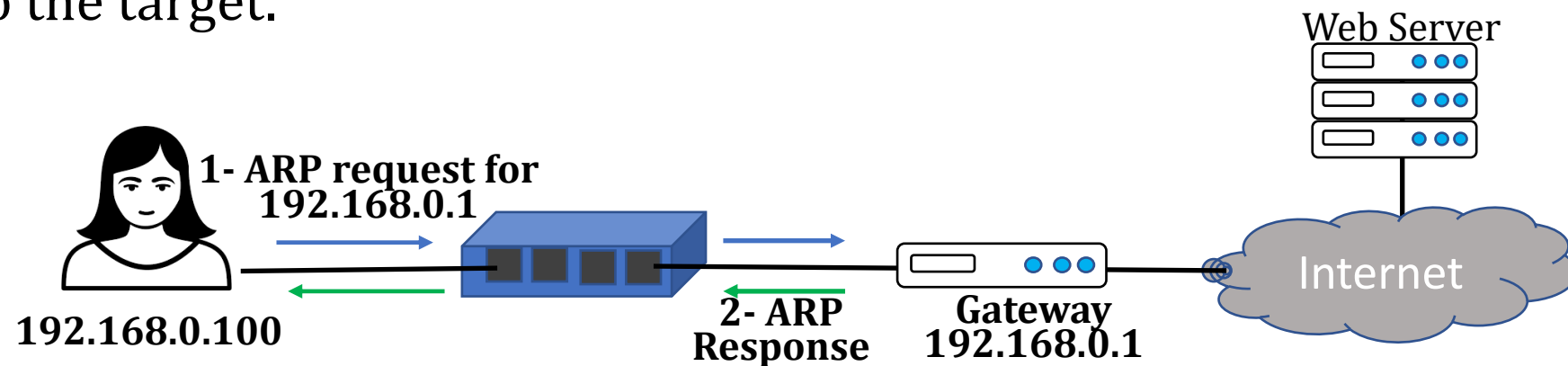
Man-in-the-Middle Attacks

- ▣ **Man-in-the-middle (MitM) is a class of attack rather than one technique.** In each type of attack, the attacker makes changes to receive packets instead of the legitimate recipient, possibly altering them, then forwards the packets to the correct destination.
- ▣ **We will give one example to illustrate how easy it can be for an attacker to spy on traffic if it is not encrypted.**
 - One MitM ARP poisoning, which we describe in the following. **ARP stands for Address Resolution Protocol.** Other attacks include DNS cache poisoning and switch table overflow.
 - Devices address each other by their MAC address. However, applications (and users) address them by **IP address or domain name (domain names being mapped to IP addresses by DNS).** ARP is the protocol that allows devices to determine the MAC address for an IP address.

Man-in-the-Middle Attacks

Consider the example in Figure below.

- ❑ Alice, is on a local subnet (IP 192.168.0.100). She connects to the Internet via a gateway (IP 192.168.0.1). They are connected by a switch.
- ❑ Alice wants to send an HTTP request to the web server. Then, Alice's computer needs to know the MAC address of the gateway. **It sends an ARP request to get MAC address of IP address 192.168.0.1. ARP request is broadcast, not unicast.** to reveal its MAC address.
- ❑ The gateway replies, and Alice's computer can send its HTTP request to the gateway, to send it on its next hop to the target.



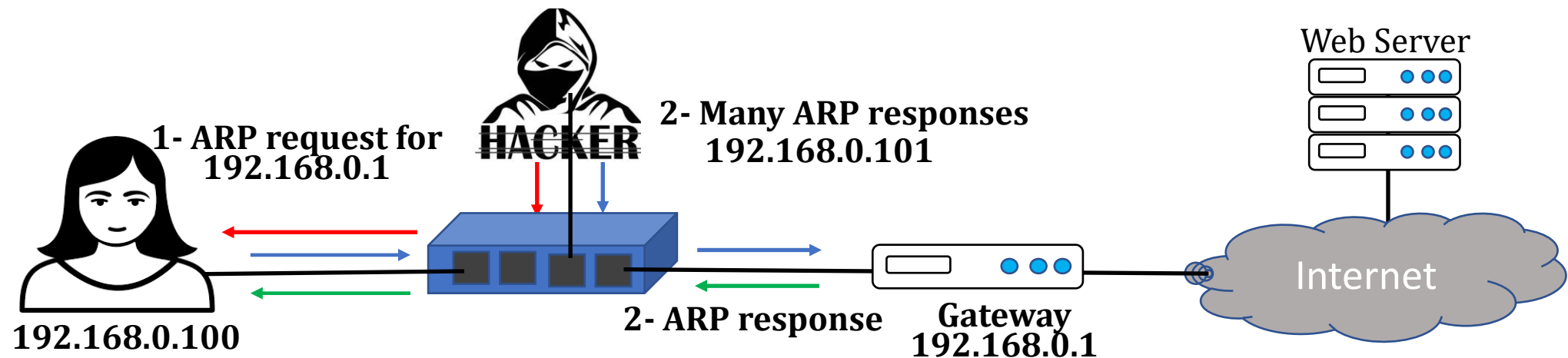


Man-in-the-Middle Attacks

- ❑ When the HTTP response comes into the gateway from the Internet, it is only addressed with an IP address. **The gateway broadcasts an ARP request, asking for 192.168.0.100 to respond with its MAC address.** Alice's computer sends back its MAC address, and the gateway can forward the response to Alice.
- ❑ An Ethernet switch knows which MAC address is on which port and only sends packets for that MAC address to that one port. **This can either be configured into the switch manually (these are known as managed switches) or it can be configured dynamically (these are unmanaged switches). In the latter, the switch acts as a hub until it knows which MAC address is on which port.** ARP requests, because they are broadcast, are always sent to all ports.
- ❑ If Alice and an attacker were on the same Ethernet hub or unmanaged switches, Alice's requests as they would reach them anyway. **An attacker also responds to the ARP request but responds with enough packets that it floods out the legitimate response.**

Man-in-the-Middle Attacks

- ❑ Consider Figure below. The hacker is at address 192.168.0.101. **When Alice's computer sends an ARP request for the MAC address of 192.168.0.1, the hacker responds with their address, 192.168.0.101.**
- ❑ If they send enough response packets, Alice will forward her request to their MAC address. They will spy on it and then forward it to the gateway.
- ❑ When the response for 192.168.0.100 arrives at the gateway, and the gateway makes an ARP request, **the attacker again responds with their MAC address. They receive the response and forward it to Alice.**





Man-in-the-Middle Attacks

- ❑ In practice, **good network administrators** and **intrusion detection systems (IDSs)** can **spot MitM attacks** because they are noisy and because more than one IP address is responding with the same MAC.
- ❑ Also, **managed switches** have features such as port security, which enforces only **one MAC address per port**, and **dynamic ARP inspection**, which filters suspicious ARP messages, mitigating the risk.
- ❑ Many smaller organizations, and most home networks, **do not have such strong defenses**, and as web developers, we cannot control what networks our sites are accessed from.
- ❑ **Wi-Fi networks can provide more MitM opportunities**. Public hotspots without WPA or WEP do not encrypt. **Attackers can also set up rogue access points.**



Defending Against Man-in-the-Middle Attacks

- ▣ We already looked at some defenses against MitM: **good IDSs and managed switches. Physical security of your network hardware is also important.**
 - **However, the best defense is to ensure you use HTTPS for all communication where trust cannot be assured. We will look at how to do this in the next section.**
- ▣ If your server allows connections over HTTP as well as HTTPS, an attacker can use a technique called **SSL stripping**. **The attacker sets themselves up as a MitM and alters the requests, switching them from HTTPS to HTTP.**
 - **Open-source software exists to not only automate this but even replace the favicon with a lock symbol to make it look encrypted. We will look at defending against this in Setting Up HTTPS.**
- ▣ Modern browsers warn users of unencrypted connections.
 - **So, another defense is to make sure your users have the latest browser installed.**



Defending Against Man-in-the-Middle Attacks

- **SSL splitting is where the attacker intercepts an HTTPS request and makes a new one to the target.**
 - **There are also valid use cases, such as intercepting corporate proxies, but nefarious ones too.**
- **When the attacker receives the response, the attacker sends their own, again over HTTPS, but using their own key.**
 - **Modern browsers lag this, popping up a warning that the TLS/SSL key does not match the domain of the sender and that it is a possible man-in-the-middle attack.**
- **Browsers require the user to make a number of clicks to confirm they are happy with the risk. Unfortunately, many users do not understand man-in-the-middle attacks.**
- **If a hacker is able to obtain the private key of a certificate authority, they will be able to sign a certificate on any chosen domain. The browser will not lag this as an attack.**
 - **This happened in 2011 when the private key of a Dutch certificate authority, DigiNotar, was compromised.**



Session Hijacking Attacks

- ▣ In earlier days of the Web, encryption was computationally and financially expensive, so not all pages were encrypted.
- ▣ Pages that asked for passwords and other sensitive information were made over HTTPS, with the rest of the application delivered with HTTP.
 - ◎ **This was true of Facebook and other popular social media services and was exploited by a tool called Firesheep. It is still available on GitHub.**
- ▣ To understand how Firesheep worked, **we must look at session cookies. Remember, the HTTP is stateless.** So that a user can be recognized between requests, a server sends a session ID cookie after successful login.
 - ◎ The session ID cookie contains a string, often simply random, that is stored in a server-side table associated with the user and client side by the browser in a cookie.
 - ◎ The browser sends the cookie with each request. The server looks the session ID up in its table and identifies the user.



Session Hijacking Attacks

- ▣ Session IDs are long and designed to be unguessable. **For this reason, the case is that no further checks are made** - if a session ID cookie matching an entry in the server table is sent, that user is logged in.
- ▣ Firesheep was released in 2010 as a Firefox extension and exploited the fact that the session ID cookie was sent by the browser in unencrypted HTTP requests.
 - *Firesheep set itself up as a **man-in-the-middle on a Wi-Fi network and sniffed packets to extract session IDs. It then presented these to the attacker in a list.***
 - *When the attacker selected an ID, it was installed as a cookie, and the attacker could use Facebook, or other services Firesheep supported, such as Twitter, as though logged in as the victim.*
- ▣ *Later a similar tool called **FaceNiff 10** was released for Android, which made the same attack even easier.*
- ▣ *These attacks are prevented by using HTTPS for all requests and responses, not just login procedures.*



Denial-of-Service Attacks

- Denial of Service, or DoS, occurs when an attacker causes a service to be unavailable for legitimate users.
 - DoS attacks are often considered the responsibility of network administrators.
 - The best-known DoS attacks are where an attack or attackers overload our network with traffic, making it unresponsive to other users.
 - Distributed denial of service, or DDoS, is a variant of this where multiple attack servers target the same service.
 - As developers, we largely rely on upstream network protection to prevent these from impacting on our servers' performance.
- However, there are classes of DoS attacks that developers can prevent or at least mitigate. *We will examine them in this section.*



The Slowloris Attack

- ❑ Web servers serve multiple simultaneous users by using **threads**. Each request is served by a thread, which cannot respond to another request until the request has finished.
 - ❑ **If a server runs 20 threads, 20 simultaneous requests can be served.** Usually, a request is served quickly.
 - *The Slowloris attack works by making a very slow request, sending one character at a time with a long pause in between each one. This keeps the thread occupied for much longer.*
 - *A single attack host can make a web server unresponsive by having as many simultaneous requests as the server has threads.*
- ❑ Apache implements timeouts for receiving requests, but the time resets each time it receives data.
 - **The mod_reqtimeout provides better protection against Slowloris attacks. The default does not protect against the attack sufficiently.**
 - **Another defense is Apache's mod_qos module. We will use this to restrict the number of connections from a single IP address.**



Consuming Back-End Resources

- ▣ The most attractive denial-of-service attacks to a hacker are those that place a heavy load on the target but not on their own resources. **The Slowloris attack is one example. The Billion Laughs attack is another.**
 - ▣ Databases and computational back ends are also attractive targets. **Imagine an applica-tion with a search function accessible by the URL**
<http://mysite.com/search?term=term&pagesize=s&start=y>
 - ◎ The URL allows the client to select how many results should be returned per page.
 - ◎ If the inputs are not sanitized and the database is very large, **an attacker can send an empty search term and a very large page size.**
 - ◎ A few simultaneous queries may be enough to make the database unresponsive. **The attacker may kill the request and start another before getting results, to avoid load on their own network.**
- ▣ **A mitigation is to cap the page size in the back-end code. Another is to disallow blank search terms and other terms that can create too much load on the database.**



Setting Up HTTPS

- ❑ The usual procedure for enabling HTTPS on your site is as follows:
 1. Create a private key that will be used to encrypt HTTPS traffic.
 2. Use the private key and your organization's details to **create a certificate signing request (CSR)**.
 3. **Send the CSR to a CA to create, sign, and send you a certificate.**
 4. Install the certificate, your private key, and the CA's certificate bundle (their certificate plus those of intermediaries).
 5. **Configure your web server to accept HTTPS and point it at the preceding files.**
 6. **Add any processes your CA provides for automating key renewal.**
 7. Restart your server.
- ❑ The exact process depends on the CA.



Installing the Certificate

How to Add an SSL Certificate to a Website

Step 1: Choose the Right Type of SSL Certificate.

Step 2: Purchase or Obtain an SSL Certificate.

Step 3: Generate a Certificate Signing Request (CSR)

Step 4: Install the SSL Certificate on Your Web Server.

Step 5: Update Your Website to HTTPS.

Step 6: Verify SSL Installation



HTTPS with Let's Encrypt

- ▣ Let's Encrypt is a nonprofit Certificate Authority providing TLS certificates to 450 million websites. (from there website)
- ▣ Let's Encrypt provides a simple process for Apache and Nginx hosts.
 - ◎ It uses the Automatic Certificate Management Environment (ACME) Protocol, an RFC that standardizes a procedure for automating domain validation, either by provisioning a DNS record under the requested domain or provisioning an HTTP record under a well-known URI within it.
- ▣ As the protocol is standardized, a number of clients can be selected to perform this task.



Creating a Self-Signed Certificate

- ❑ **For development purposes, we cannot use Let's Encrypt, or any other CA,** to generate a certificate as it is bound to one or more public DNS domain names **(though an ACME server has been developed for this purpose).**
- ❑ **Larger organizations often have their own CA to generate certificates** for sites on their intranet. If we want to explore using HTTPS in an internal development environment, **we need to sign our own certificate and configure our web browsers to accept it.**
- ❑ **Creating a self-signed key is like creating a CA-signed key.** The difference is we use our own private key to sign the certificate - we are our own CA.



Browser Requirements for Self-Signed Certificates

- ❑ Browsers implement varying levels of security regarding self-signed certificates. Firefox allows the user to add an exception, allowing the site to be visited. **It still lags the site as unsafe but does use encryption.**
- ❑ Chrome does not allow the user to add an exception, and the certificate must be manually added. **It also requires a Subject Alternative Name (SAN) to be added to the certificate. This is a way of specifying multiple domain names and/or IP addresses in the same certificate.**
- ❑ Even if we have only one IP address or domain name, Chrome still requires a SAN record. **We included the details in the v3.ext file as OpenSSL does not support these parameters on the command line.**



Reverse Proxies and Tunnels

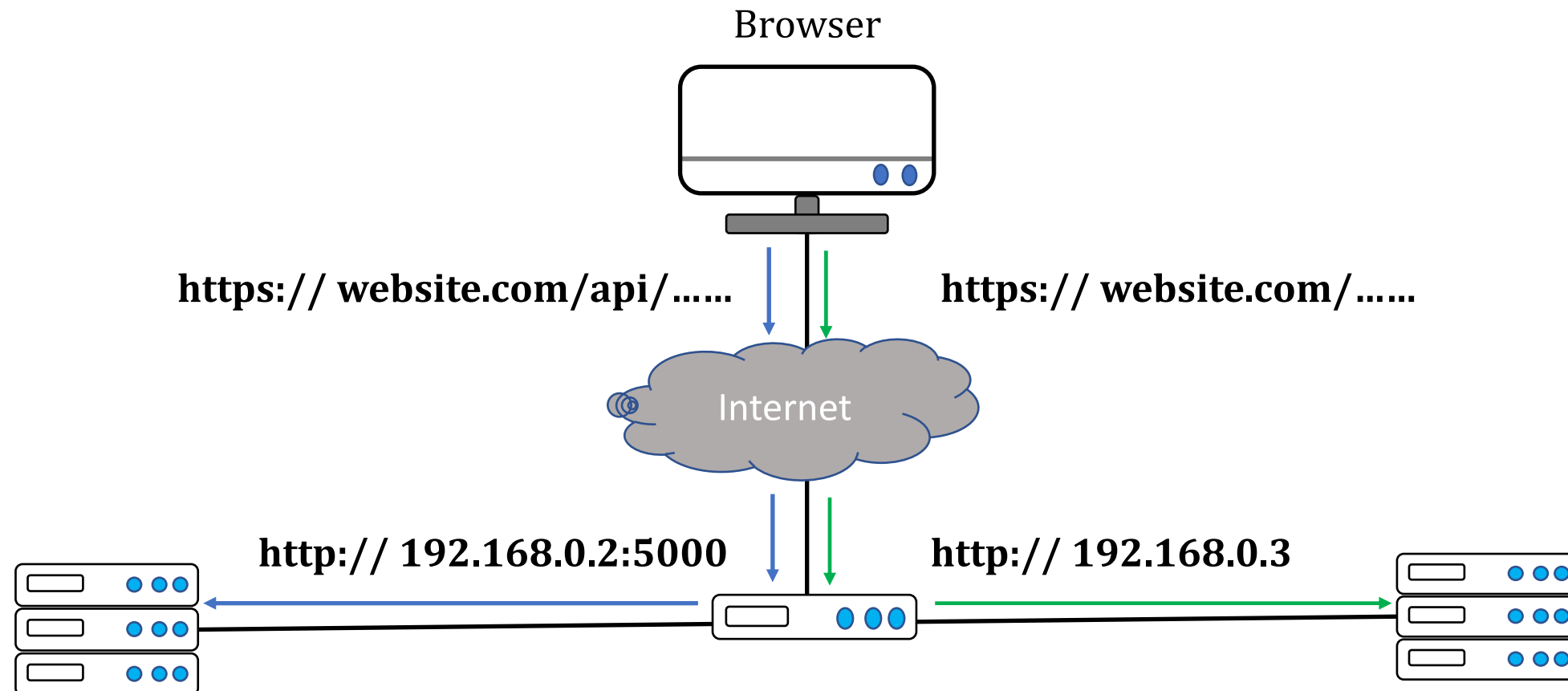
Reverse Proxies

- ❑ A **forward proxy** sits **between a browser and the Internet**. Requests from, and responses to, the browser are tunnelled through the proxy.
- ❑ A **reverse proxy** sits **between a server and the Internet**. Requests to, and responses from, the server are tunnelled through the proxy.
- ❑ A reverse proxy is used for a number of reasons, for example:
 - To **hide architecture** by sending all traffic through a single host
 - To **open firewalls** to fewer hosts
 - Load balancing
 - To **provide users with a default port** (80 or 443) when services run on a different one (e.g., a REST API on port 5000)
 - To **provide encryption** for services that don't implement it, or for which SSL/TLS certificates cannot be obtained
 - To **require authentication** to services that don't implement it



Reverse Proxies and Tunnels

- ❑ The Figure below shows a **reverse proxy** in front of a web server and a REST API.
- ❑ URLs starting with `/api` are directed to <http://192.168.0.2:5000/...> with the `/api` dropped from the URL. All other requests are directed to <http://192.168.0.3/...>.





Reverse Proxies and Tunnels

In this example, we are assuming **no trust boundaries between the three servers, so HTTP is used instead of HTTPS.**

- ❑ If this is not the case, HTTPS can be used, but certificates need to be obtained for 192.168.0.2 and 192.168.0.3.
- ❑ If, as in this example, these IP addresses are internal, public CAs cannot issue certificates. In this case, an organization can issue its own internal certificates and add its internal CA to the reverse proxy's list of root authorities.
- ❑ Alternatively, it can use self-signed certificates. They are not sent to browsers, only internally between servers.



Reverse Proxies

- **There are a number of advantages with this architecture:**
 - ▣ **The hosts on 192.168.0.* do not have to implement HTTPS (unless they cross a trust boundary, as discussed previously).**
 - ▣ **The hosts on 192.168.0.* do not have to be exposed outside the intranet.**
 - ▣ **The user is unaware of the names or even existence of these hosts.**
 - ▣ **The reverse proxy can be made a load balancer and additional back-end servers added.**
 - ▣ **We can change the address of the servers, or the ports the services run on, and only need to update the reverse proxy.**
- **The disadvantage is that all traffic passes through a single proxy, creating a bottleneck if not sized correctly.**



SSH Tunnels

- ▣ HTTPS only provides encryption for HTTP. It cannot be used to encrypt database connections. Many other protocols, such as, the Postgres, do also offer encryption. **For protocols that do not, SSH tunnels can be used.**
- ▣ An SSH tunnel, known as **SSH port forwarding**, wraps an arbitrary TCP/IP protocol in **an encrypted SSH connection**. There are two types:
 - ▣ *local forwarding* and
 - ▣ *remote forwarding*.
- ◎ In both cases, an SSH port is opened to listen to connections. Upon connection, a tunnel is established and used to forward packets to the target port and host.

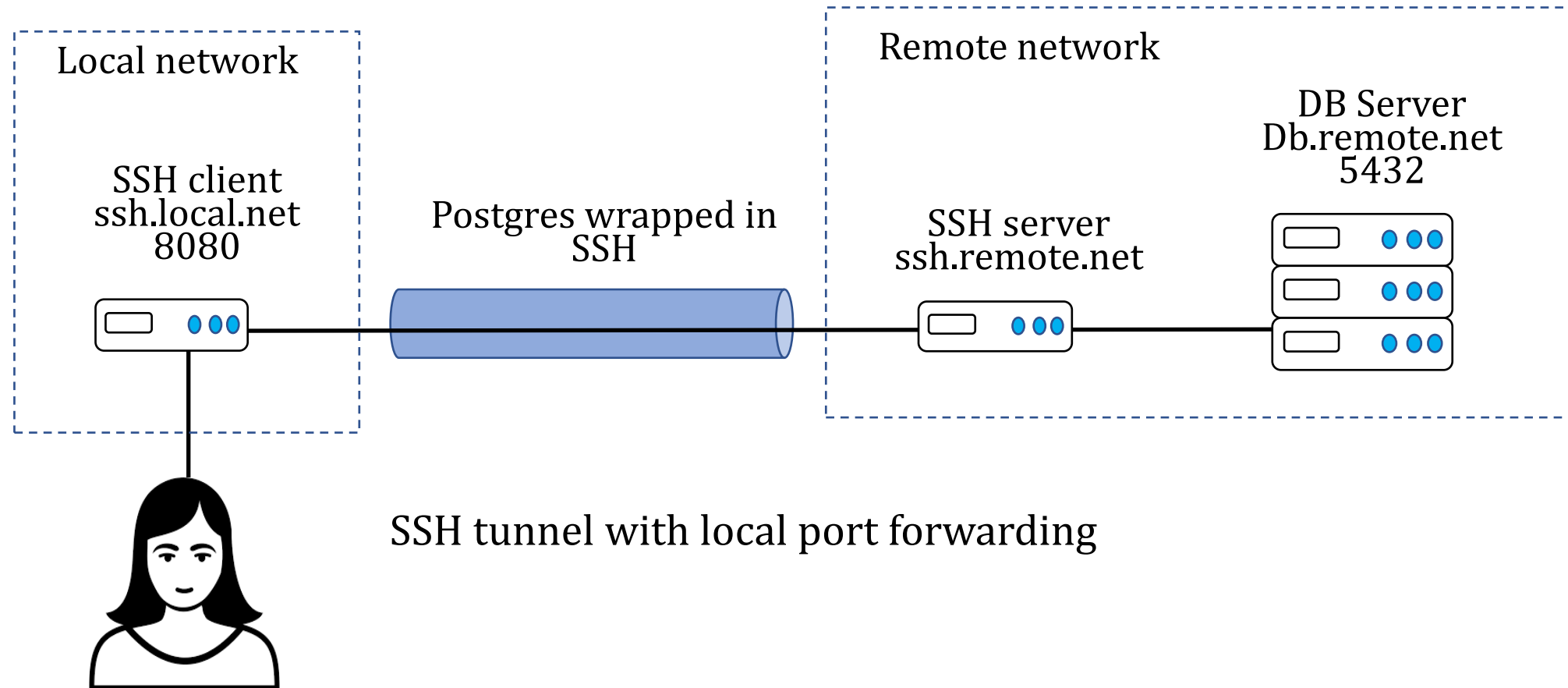


SSH Tunnels

The Figure below shows an example for **local port forwarding**.

- ▣ The SSH server daemon is running on host **ssh.remote.net**.
- ▣ The developer wants to establish a secure connection to **Postgres running on port 5432 of host db.remote.net**.
- ◎ They pick a host local to them that they can run an SSH client on. This may be localhost. In our example, it is client.remote.net. They pick a local port: in our example, 8080.
- ◎ They use the SSH client to connect to the SSH daemon running on **ssh.remote.net**. When they connect to port 8080 on client.remote.net, say, with the **psql** command, the Postgres packets will be wrapped in SSH and sent to **ssh.remote.net**, which will unwrap them and send them onto the database server, **db.remote.net**, on port 5432.

SSH Tunnels



- ❑ *The user needs an account on `ssh.remote.net` and to have permission to connect to its SSH daemon.*



SSH Tunnels

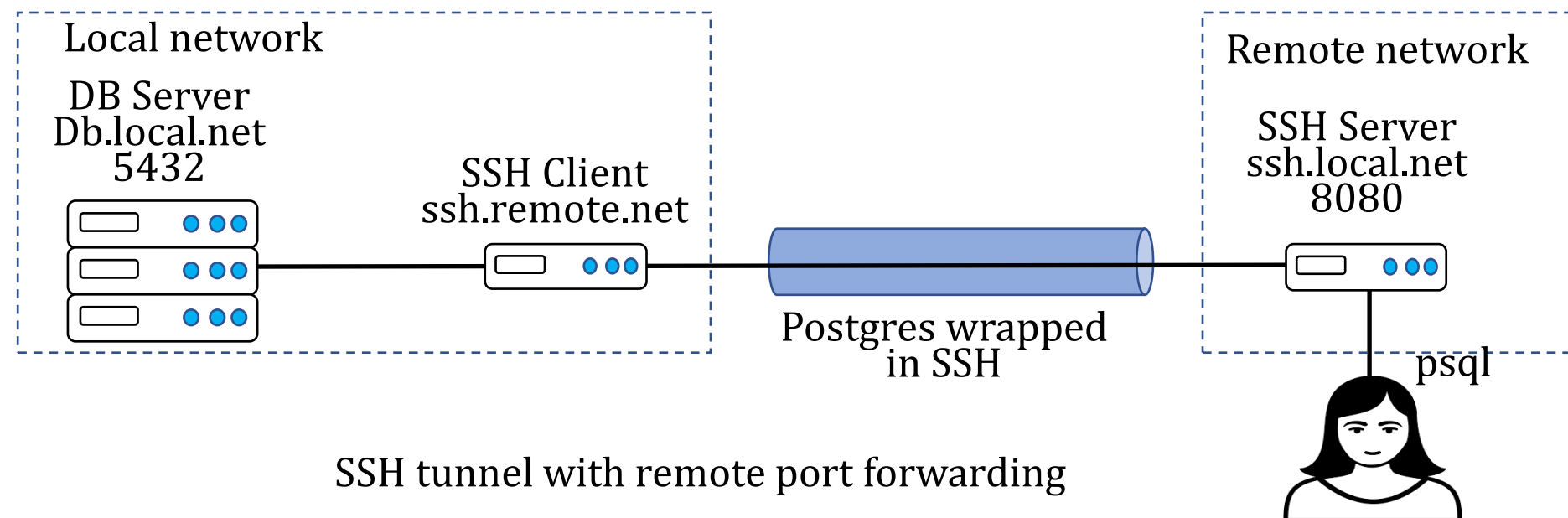
- ❑ It should be noted that only the connection from `client.local.net` to `ssh.remote.net` is encrypted, **not from `ssh.remote.net` to `db.remote.net`**. It should therefore not cross a trust boundary.
- ▣ One solution is to run the SSH server on the same host as the database.
- ❑ **Local port forwarding is useful for encrypting traffic over an untrusted channel when the protocol does not support TLS/SSL itself.**
- ❑ It is also useful for bypassing a firewall - in the preceding example, port 5432 does not have to be open outside the remote network.



Installing and Configuring Services

SSH Tunnels

- ❑ **Remote port forwarding** allows a user to open a port on a local host even if it is not accessible from outside. As illustrated in the Figure below.
 - *Say the owner of a database wants to allow outside users to access their Postgres server on db.local.net.*
 - *They establish a remote port forwarding tunnel to an SSH server running on a remote network, ssh.remote.net, on which they have an account. Now an outside user can connect to ssh.remote.net, and their connection will be forwarded to db.local.net.*





Installing and Configuring Services

❑ Hiding Service Details

- ❑ We looked at hiding hostnames and ports behind a reverse proxy in the last section. This is good practice because it reduces your attack surface, especially if the back-end hosts are behind a firewall.
- ❑ Hackers learn about their target systems by reconnaissance. One open source tool for this is nmap, which probes hosts and their open ports. We will not discuss nmap in detail, but one useful function it performs is fingerprinting:
 - determining the names and versions of services running on open ports.
 - If an attacker knows the name and version of a service you are running, they can look up its known vulnerabilities.
- ❑ You can reduce the information nmap and similar tools can be obtained by
 - ⦿ Blocking access to ports from all unnecessary hosts using a corporate firewall, host firewall, or TCP Wrappers removing banners from services.
 - ⦿ Banners are text that a service displays when it is connected to. For example, you can see that the web server is Apache, and which version it is, by running the command.



Server Configuration

❑ Host Firewalls

- ◎ Firewalls protect ports for whole networks or subnets. A **host firewall** runs on a host and protects just that host's ports.
- ◎ **Ubuntu** comes with **ufw firewall** that controls Netfilter for packet filtering.
- ◎ **Windows** also comes with **Microsoft Defender Firewall**.
- ◎ Ports can be allowed or denied for all connections, for specific IP addresses, for IP ranges, and for interfaces. Service names (e.g., http) can be given in place of port numbers.
- ◎ **Rules are executed from top to bottom.** When a connection is attempted, Netfilter checks if the first rule applies. If it does, and it is an ALLOW, the connection is allowed. If it matches and is a DENY, the rule is denied. Otherwise, Netfilter proceeds to the next rule.
- ◎ **If no rules match, the default rule applies.** The default case sets to allow all outgoing connections and deny all incoming.



Server Configuration

❑ TCP Wrappers

- ⦿ TCP Wrappers provides application-level filtering. Applications support TCP Wrappers by linking against its library `tcpwrap.a` or `tcpwrap.so`.
- ⦿ Configuration is in the files `/etc/hosts.allow` and `hosts.deny`.
- ⦿ The `hosts.allow` and `hosts.deny` files consist of lines in the form `daemon-list : client-list [: option : option ...]`
 - ⦿ **daemon-list** is a comma-separated list of application names (not service names), or the wildcard `ALL`.
 - ⦿ **client-list** is a comma-separated list of hostnames, IP addresses, hostname/IP address patterns, and wildcards. **option** is an optional action such as spawning a program.



❑ TCP Wrappers

- The `hosts.allow` and `hosts.deny` files are designed to be used together. When a connection is attempted, `tcpwrap` looks at the first line of `hosts.allow`.
 - ⦿ If it matches, the connection is allowed, and actions on that line are performed. If not, `tcpwrap` proceeds to the next line.
- If no lines match, the `tcpwrap` library looks at the first line in `hosts.deny`.
 - ⦿ If it matches, the connection is denied, and any actions are performed. If not, it moves to the next line. If no lines match, the connection is allowed.
- To disallow every connection by default, add the following at the end of the `hosts.deny`: **ALL : ALL**



Hiding Errors

- ❑ In developing a web application, we need to see errors. **Frameworks can display back-end errors and stack traces in the web page, aiding the development process.**
- ❑ If left in production systems, error messages can give away details that help attackers find vulnerabilities.

Custom 404 and 500 Pages

- ❑ Removing debugging output reduces leakage of program details. **We can reduce this further by customizing the web server's default 404 and 500 error pages.**
- ❑ Web server returns a 404 Not Found response code when a requested URL is not found. The text in the body of the response can be configured by your web server or WAF. Similarly, an error executing back-end code returns a default 500 Server Error page.
- ❑ Both of these can be overridden to give friendlier yet vaguer errors about what went wrong



Database Configuration

- ❑ Web applications often use databases to persist data. If databases is misconfigured, vulnerabilities can be introduced.
- ❑ A misconfigured database can result in the database user's password being disclosed to hackers and unauthorized users. It can enable attackers to gain shell access as the database OS user.
- ❑ Finally, it can allow attackers who have obtained the password to get SQL access via a command-line or database admin tool.



Database Password Management

- ❑ Storing passwords in a file as cleartext is generally discouraged, it is common practice for databases as it is inconvenient to enter it manually each time the server starts.
- ❑ To minimize the impact of vulnerabilities, the database user should only have as many permissions as are needed to run the application. *This usually means that users should not be able to change the database schema, or create or drop databases users.*
- ❑ Database superuser passwords should never be stored on the filesystem. **The password should be stored in a file that as few users as possible have access to.** It needs to be readable by the user the web server runs as (e.g., www-data) but not be world readable.
- ❑ One approach is to make the password file owned by the account the software is installed as (user prod). *Make the file containing the password owned by prod with group www-data and give the file permission 640 (read-write for user prod, read-only for group www-data, no permissions for other).*



Access to the Database Host and Port

- ▣ If the database port is accessible to outside hosts, an attacker who does obtain the password may be able to connect from outside your network using a command-line tool such as Postgres' psql, mssql-cli, etc. He may also attempt to guess the password by brute force.
- ▣ It is better to restrict access to the database port to only hosts that need it for operation. Options include:
 - ⊙ Blocking the port with a host firewall or TCP Wrappers
 - ⊙ Blocking access with database-specific configuration
- ▣ If your database runs on the same host as the web server, and operations staff can perform maintenance from the same host, then TCP/IP access to the database may not be needed at all. In this case, you can disable it in the database configuration or block it by firewall.



Access to the Database Host and Port

- ☐ If your database is only accessed over a private network, e.g., 192.168.0.*, access can be restricted to this IP range to reduce the attack surface.
- ☐ If database connections must cross trust boundaries, many database servers offer encryption. If your database server does not offer encryption, SSH tunneling can be used.



Web-Based Administration Consoles

- ▣ Web applications often use databases to persist data. If misconfigured, vulnerabilities can be introduced. **Database servers often provide web-based administration consoles.** Third-party offerings also exist.
 - ◎ One **advantage** of these tools is that they can be served from the database server itself, avoiding the need to open the database port to other hosts.
 - ◎ The **disadvantage** is anyone with a web browser on a host not blocked by a firewall can attempt to connect to the database.
- ▣ Running a web-based admin console shifts the threat of unauthorized access from the database port to the web interface. Care should be taken to ensure this interface is secure.
 - ◎ **For example, it should only support HTTPS, permanently redirecting port 80 to 443.**



Securing the File System

- ▣ We intentionally expose part of the filesystem so that
 - Users can view files from the public HTML path (HTML files, media, JavaScript, etc.) through a web browser
 - Developers and operational staff can make changes
- ▣ We look at ensuring we don't expose more than we intend.
- ▣ The site configuration file also defines the location of error and access log files. **These should not be within the public path; otherwise, web users will be able to view them through their web browser.**
- ▣ In almost all cases, the public path directories and the files within them should not be writable by the user the web server runs as (**www-data** in our case).
- ▣ If a vulnerability exists that allows an attacker access to the filesystem through the web server (**e.g., a command injection vulnerability**), and **www-data** can write these files, then the attacker can change your application and potentially execute code on your server.



Securing the File System

- ❑ Any directories that are in the public path should either have an index.html file or else the directive Options -Index should be included in the <Directory> block. **If it is not, then the user will be able to perform a directory listing, viewing the directory's contents. This can expose vulnerabilities.**
- ❑ Make sure that any files that are not part of your application are also not in the public path.
 - For example, old files that are no longer used. HTML, CSS, media, PHP, and JavaScript files can be found by attackers even if there are no links to them by using brute-force tools such as Dirbuster.
- ❑ This is also true of URLs, such as REST API calls, that are no longer needed for the application and development-related directories. **Be sure to delete debugging URLs from productive systems.**



Code Directories

- ❑ Web applications usually contain code. **Web servers should be configured to only serve code by executing it, not displaying it as plain text.** If users can see the code, they may discover vulnerabilities they can exploit.
- ❑ The original way of serving code from a web server is by putting executables or scripts in a cgi-bin directory. **This is configured placing a ScriptAlias and Options +ExecCGI directives within it.** If cgi-bin directories are used, care should be taken that the `<Directory>` block is set up correctly so that the files cannot be served as plain text.
- ❑ As with HTML directories, cgi-bin and WSGI directories, and the files within them, should not be writable by the user.



Upload Directories

- ❑ Sometimes, a directory has to be writable so that users can upload files, e. g, uploading images on social media sites. **Allowing users to change the content on your site can introduce vulnerabilities.**
- ❑ The following principles reduce the risk:
 1. **Keep the upload directories out of the public path so that the files cannot be accessed directly by a URL.**
 2. **Restrict the file types and check that the file conforms to the type.**
 3. **Scan the file for malware.**
 4. **Restrict file size to avoid denial-of-service attacks.**
 5. **Beware of file types that can contain harmful features such as Microsoft Office documents with macros.**
 6. **Require users to authenticate and ensure that they can only upload to a directory dedicated to them. This avoids one user overriding files of another.**
 7. **Economize error messages and don't reveal the upload path.**



Secrets

- Web applications contain some files with particularly sensitive data, for example: **Database passwords and Private keys. We collectively call these files secrets.** They are installed in various directories and need to have differing permissions and ownership.
- Database passwords need to be readable, but not writable, by the web server user.
- Private keys should only be readable by root.
- TLS certificates can be world readable but not writable.
- The script containing the environment variable needs to be readable but not writable by the web server user.
- An easy way to ensure the permissions remain correct is to keep all secrets in a single directory and system link them to the correct locations.
- Care should also be taken to keep production values for secrets out of source code control and development hosts.